

returning from shared object...

returning to main...

freeing memory...

```
[root@workbenchsvr malloc_hook]#
```

So, let's take a closer look at our first hook. The *dlsym()* function takes two parameters: the first is a handle returned by *dlopen()*. Here, we must use *RTLD_NEXT* for function interposition. This tells the dynamic linker to find the next reference to the specified function, not the one that is calling *dlsym()*. The second parameter is the symbol name ('*malloc*', in this case), as a character string. *dlsym()* returns the address of the symbol specified as the second parameter. While compiling, *fPIC* is used to create a position-independent object.

The *LD_PRELOAD* environment variable gives the loader a list of libraries to load before anything else. Our use of it will load *libprog2.so* and dynamically link it with the *prog1* binary. Don't forget to provide an absolute path to the *.so*, in *LD_PRELOAD*. And, of course, include *_GNU_SOURCE* if you want to use certain extensions while using the GNU C library, because some extensions may not be available on other non-GNU systems and adding this *#define* will increase portability.

Can we hook every function with *dlsym()*?

The above hook method will not work if you want to wrap *dlsym()* itself, or wrap any library functions that internally call *dlsym()*. So, is there any way to interpose *dlsym()*? Yes, but you can't use the same hook procedure—check the output if you try that. First, create a shared object *libfile.so* from *file1.c* and *file2.c* (below). Then compile it with *gcc -rdynamic -o dl_prog1 dl_prog1.c -ldl*. Yes, the output is obvious:

```
/* file1.c */
void file1(int *i)
{
    *i=100;
}

/* file2.c */
void file2(int *i)
{
    *i=200;
}
```

The following *dl_prog1.c* is a simple program to show the functionality of *dlopen()* and *dlsym()*. The functions *file1()* and *file2()* are defined in *file1.c* and *file2.c*.

```
#include<stdio.h>
#include<dlfcn.h>
#include<stdlib.h>

void file1(int *i);
```

```
void file2(int *i);
```

```
int main(void)
```

```
{
    void *handler;

    int (*fn) (int *);

    int x;

    char *error;

    handler = dlopen("/home/dibyendu/dlsym_hook/libfile.so",
RTLD_LAZY);

    if (!handler)

    {

        fprintf(stderr, "%s\n", dlerror());

        exit(1);

    }

    fn = dlsym(handler, "file1");    /* getting the handle of
file1 through dlsym() */

    if ((error = dlerror()) != NULL) /* checking error
through dlerror() */

    {

        fprintf(stderr, "%s\n", error);

        exit(1);

    }

    (*fn)(&x);    /* Calling file1()
to resolve x */

    printf("The value of x is %d\n", x);

    dlclose(handler);    /* closing the file
handle */

    return 0;

}
```

```
[root@workbenchsvr dlsym_hook]# gcc -shared -ldl -fPIC file1.c
file2.c -o libfile.so
```